NoSQL Database Design – Lab Sheet 5

Questions, sample commands/queries, and expected outputs

Important: Riak KV questions use HTTP API examples with `curl` (assumes Riak is installed and reachable at localhost:8098). Neo4j questions use Cypher in Neo4j Browser. Outputs are illustrative examples; exact responses may vary by version and setup.

1. Install/configure Riak KV and perform basic student-record key-value operations.

Code / Query

```
# Linux installation is package/version dependent. After installing Riak:
sudo systemctl start riak
curl -i http://localhost:8098/ping

# Store a student record
curl -i -X PUT http://localhost:8098/riak/students/STU101 \
  -H 'Content-Type: application/json' \
  -d '{"name":"Aman","course":"BCA","year":3}'

# Retrieve, update, then delete
curl -i http://localhost:8098/riak/students/STU101
curl -i -X PUT http://localhost:8098/riak/students/STU101 \
  -H 'Content-Type: application/json' \
  -d '{"name":"Aman","course":"BCA","year":4}'
curl -i -X DELETE http://localhost:8098/riak/students/STU101
```

Expected output

```
GET /ping -> 200 OK
GET /riak/students/STU101 -> 200 OK
{"name":"Aman","course":"BCA","year":3}
Updated record -> {"name":"Aman","course":"BCA","year":4}
DELETE -> 204 No Content
```

Note: Some Linux distributions require a different installation method. Check the Riak service name and API port for your installation.

2. Create a user_sessions bucket and retrieve session information by key.

Code / Query

```
curl -i -X PUT http://localhost:8098/riak/user_sessions/SESSION9001 \
  -H 'Content-Type: application/json' \
  -d '{"session_id":"SESSION9001","user_id":"U102","login_time":"2026-10-16T09:00:00","status":"active"}'

curl -i http://localhost:8098/riak/user_sessions/SESSION9001
```

Expected output

```
PUT -> 204 No Content
GET -> 200 OK
{"session_id":"SESSION9001","user_id":"U102","login_time":"2026-10-16T09:00:00","status":"active"}
```

Note: Classic Riak KV HTTP APIs may use bucket types and bucket properties; ensure the bucket is available in your setup.

3. Store user profile and preferences and demonstrate PUT and GET.

Code / Query

```
curl -i -X PUT http://localhost:8098/riak/user_profiles/U102 \
  -H 'Content-Type: application/json' \
  -d '{"name":"Riya","email":"riya@example.com","city":"Roorkee","language":"English","theme":"light","notifications":true}'

curl -i http://localhost:8098/riak/user_profiles/U102
```

Expected output

```
PUT -> 204 No Content
GET -> 200 OK
{"name":"Riya","email":"riya@example.com","city":"Roorkee","language":"English","theme":"light","notifications":true}
```

4. Implement a shopping cart: add, update, retrieve, and remove products.

Code / Query

```
# Save cart with two products
curl -i -X PUT http://localhost:8098/riak/carts/U102 \
  -H 'Content-Type: application/json' \
  -d '{"items":[{"product_id":"P1","qty":2,"price":100},{"product_id":"P2","qty":1,"price":250}],"total":450}'
```

```
# Update quantity/total (replace the cart value)
curl -i -X PUT http://localhost:8098/riak/carts/U102 \
  -H 'Content-Type: application/json' \
  -d '{"items":[{"product_id":"P1","qty":3,"price":100},{"product_id":"P2","qty":1,"price":250}],"total":550}'

curl -i http://localhost:8098/riak/carts/U102

# Remove P2 by saving the revised cart
curl -i -X PUT http://localhost:8098/riak/carts/U102 \
  -H 'Content-Type: application/json' \
  -d '{"items":[{"product_id":"P1","qty":3,"price":100}],"total":300}'
```

Expected output

```
Initial cart: total = 450
Updated cart: total = 550
GET -> {"items":[{"product_id":"P1","qty":3,"price":100},{"product_id":"P2","qty":1,"price":250}],"total":550}
After removing P2: {"items":[{"product_id":"P1","qty":3,"price":100}],"total":300}
```

Note: Riak KV stores the value for a key; application logic updates the JSON document and calculates totals.

5. Demonstrate strings, JSON documents, and lists; explain bucket, key, and value.

Code / Query

```
# String value
curl -i -X PUT http://localhost:8098/riak/demo/string1 -H 'Content-Type: text/plain' --data 'Hello Riak'
curl http://localhost:8098/riak/demo/string1

# JSON document
curl -i -X PUT http://localhost:8098/riak/demo/student1 -H 'Content-Type: application/json' --data '{"name":"Neha","marks":8
curl http://localhost:8098/riak/demo/student1

# A list stored as JSON
curl -i -X PUT http://localhost:8098/riak/demo/topics -H 'Content-Type: application/json' --data '["SQL","Python","NoSQL"]'
curl http://localhost:8098/riak/demo/topics
```

Expected output

```
string1 -> Hello Riak
student1 -> {"name":"Neha","marks":88}
topics -> ["SQL","Python","NoSQL"]

Bucket = collection/namespace; key = identifier; value = stored data.
```

6. Demonstrate read/write quorum settings and consistency.

Code / Query

```
# Example: inspect bucket properties (endpoint availability depends on Riak version)
curl -i http://localhost:8098/riak/buckets/students/props

# Read/write with explicit quorum parameters where supported
curl -i -X PUT 'http://localhost:8098/riak/students/STU202?w=2&dw=2' \
  -H 'Content-Type: application/json' -d '{"name":"Kabir","course":"BCA"}'
curl -i 'http://localhost:8098/riak/students/STU202?r=2'
```

Expected output

```
Bucket properties endpoint returns bucket configuration (JSON).
PUT -> 204 No Content
GET -> 200 OK
{"name":"Kabir","course":"BCA"}

r controls read quorum; w controls write quorum; dw requests durable writes.
```

Note: Exact supported parameters and quorum behavior depend on Riak version and cluster size. A single-node setup cannot demonstrate distributed consistency fully.

7. Demonstrate scaling using multiple buckets and many records.

Code / Query

```
# Insert 3 sample records; repeat this pattern for a larger dataset
for i in 1 2 3; do
  curl -s -X PUT "http://localhost:8098/riak/students/STU$i" \
    -H 'Content-Type: application/json' \
    -d "{\"id\":\"STU$i\",\"course\":\"BCA\"}"
done

curl -s -X PUT http://localhost:8098/riak/courses/C101 \
  -H 'Content-Type: application/json' -d '{"name":"NoSQL"}'
curl -i http://localhost:8098/riak/students/STU2
```

Expected output

```
Three student keys and one course key are stored.
GET /riak/students/STU2 -> {"id":"STU2","course":"BCA"}

Riak partitions data across the cluster using consistent hashing; adding nodes can increase capacity.
```

Note: To demonstrate horizontal scaling, use a multi-node cluster and monitor data distribution; localhost alone does not prove distribution.

8. Create a transaction-like shopping scenario and discuss transaction limitations.

Code / Query

```
# Step 1: add an item
curl -s -X PUT http://localhost:8098/riak/carts/U205 -H 'Content-Type: application/json' \
  -d '{"items":[{"product_id":"P10","qty":1,"price":200}],"total":200}'

# Step 2: update quantity and total
curl -s -X PUT http://localhost:8098/riak/carts/U205 -H 'Content-Type: application/json' \
  -d '{"items":[{"product_id":"P10","qty":2,"price":200}],"total":400}'

curl -s http://localhost:8098/riak/carts/U205
```

Expected output

```
{"items":[{"product_id":"P10","qty":2,"price":200}],"total":400}

This is an application-level sequence, not a multi-key ACID transaction. If one operation fails, the application must handle
```

Note: Classic Riak KV does not provide general multi-key ACID transactions like a relational database.

9. Implement query-by-data for user records using application-level filtering.

Code / Query

```
# Store two user JSON records
curl -s -X PUT http://localhost:8098/riak/users/U1 -H 'Content-Type: application/json' \
  -d '{"name":"Asha","city":"Roorkee","age":21,"preference":"music"}'
curl -s -X PUT http://localhost:8098/riak/users/U2 -H 'Content-Type: application/json' \
  -d '{"name":"Dev","city":"Haridwar","age":22,"preference":"sports"}'

# Retrieve known keys; an application can parse records and filter by city/age.
curl -s http://localhost:8098/riak/users/U1
curl -s http://localhost:8098/riak/users/U2
```

Expected output

```
U1 -> {"name":"Asha","city":"Roorkee","age":21,"preference":"music"}
U2 -> {"name":"Dev","city":"Haridwar","age":22,"preference":"sports"}

Application filter city == "Roorkee" -> Asha
```

Note: Riak KV is key-oriented; attribute searches require suitable secondary-index/search features or application-maintained lookup logic.

10. Demonstrate set operations for interests or product categories.

Code / Query

```
# Store a set-like JSON array
curl -s -X PUT http://localhost:8098/riak/interests/U1 -H 'Content-Type: application/json' \
  -d '["music","travel","reading"]'
curl -s http://localhost:8098/riak/interests/U1

# Application-level add/remove and intersection:
# A = ["music","travel","reading"]; B = ["travel","sports"]
# Add "coding" to A, remove "reading" from A, then compute intersection(A,B).
```

Expected output

```
Stored A = ["music","travel","reading"]
After add coding and remove reading: ["music","travel","coding"]
Intersection with B = ["travel"]
```

Note: These are application-level set operations on a JSON array. Riak Data Types also support set semantics when configured and used through the appropriate API.

Neo4j and Cypher

Run the following queries in Neo4j Browser. The outputs show representative results.

11. Install/configure Neo4j and create Student, Course, and Department nodes with relationships.

Code / Query

```
CREATE (s:Student {id:'S1', name:'Aman'})
CREATE (c:Course {code:'BCA', name:'Bachelor of Computer Applications'})
CREATE (d:Department {name:'Computer Science'})
CREATE (s)-[:ENROLLED_IN]->(c)
CREATE (c)-[:OFFERED_BY]->(d)
RETURN s, c, d;
```

Expected output

```
3 nodes and 2 relationships created.
Returned graph:
(Student {id:'S1', name:'Aman'})
  -[:ENROLLED_IN]-> (Course {code:'BCA', name:'Bachelor of Computer Applications'})
  -[:OFFERED_BY]-> (Department {name:'Computer Science'})
```

12. Create a social-network graph and find a person's friends and followers.

Code / Query

```
CREATE (a:Person {name:'Asha'})
CREATE (b:Person {name:'Ravi'})
CREATE (c:Person {name:'Meera'})
CREATE (a)-[:FRIENDS_WITH]->(b)
CREATE (c)-[:FOLLOWS]->(a)
CREATE (a)-[:WORKS_AT]->(:Company {name:'ABC Tech'});

MATCH (p:Person {name:'Asha'})-[:FRIENDS_WITH]->(friend)
RETURN friend.name AS friend;

MATCH (follower:Person)-[:FOLLOWS]->(p:Person {name:'Asha'})
RETURN follower.name AS follower;
```

Expected output

```
Friends query:
friend
"Ravi"

Followers query:
follower
"Meera"
```

13. Create an e-commerce graph and find products purchased by a customer.

Code / Query

```
CREATE (c:Customer {id:'C1', name:'Riya'})
CREATE (p:Product {id:'P1', name:'Laptop'})
CREATE (cat:Category {name:'Electronics'})
CREATE (o:Order {id:'O1', date:'2026-10-16'})
CREATE (c)-[:PLACED]->(o)
CREATE (o)-[:CONTAINS {quantity:1}]->(p)
CREATE (p)-[:IN_CATEGORY]->(cat);

MATCH (c:Customer {name:'Riya'})-[:PLACED]->(:Order)-[:CONTAINS]->(p:Product)
RETURN p.name AS purchased_product;
```

Expected output

```
purchased_product
"Laptop"
```

14. Demonstrate CREATE, MATCH, SET, and DELETE in a transaction.

Code / Query

```
// Run in Neo4j Browser
BEGIN
CREATE (p:Person {name:'Nisha', city:'Roorkee'})
WITH p
SET p.city = 'Haridwar'
RETURN p.name AS name, p.city AS city;
COMMIT
```

```
// Read updated data
MATCH (p:Person {name:'Nisha'}) RETURN p.name, p.city;

// Delete the sample node
MATCH (p:Person {name:'Nisha'}) DELETE p;
```

Expected output

```
Inside transaction:
name   city
"Nisha" "Haridwar"

After COMMIT, MATCH returns Nisha with city = Haridwar.
After DELETE, the sample node is removed.
```

Note: Explicit BEGIN/COMMIT support depends on the Neo4j interface/version; in Neo4j Browser, each submitted query normally runs as a transaction. For a simple Browser lab, execute CREATE/SET/RETURN, then MATCH, then DELETE as separate queries.

15. Build a small Neo4j application example with CRUD and relationship-based queries.

Code / Query

```
// CREATE
CREATE (u:User {id:1, name:'Aman', city:'Roorkee'})
CREATE (r:Resource {name:'Python Course'})
CREATE (u)-[:RECOMMENDS]->(r);

// READ
MATCH (u:User {id:1})-[:RECOMMENDS]->(r)
RETURN u.name AS user, r.name AS recommendation;

// UPDATE
MATCH (u:User {id:1}) SET u.city='Haridwar' RETURN u;

// DELETE the example relationship and nodes
MATCH (u:User {id:1})-[rel:RECOMMENDS]->(r:Resource {name:'Python Course'})
DELETE rel;
MATCH (u:User {id:1}) DETACH DELETE u;
MATCH (r:Resource {name:'Python Course'}) DETACH DELETE r;
```

Expected output

```
READ result:
user    recommendation
"Aman"  "Python Course"

UPDATE result:
User {id:1, name:'Aman', city:'Haridwar'}

DELETE removes the relationship and then the sample User and Resource nodes.
```

Note: Relationship queries are useful for social networks, recommendations, fraud detection, and knowledge graphs. For a Python application, use the official Neo4j Python driver and connect with your database URI and credentials.

Quick reminders

• Riak KV: bucket + key identifies a stored value; PUT stores/replaces a value, GET retrieves it, DELETE removes it.

• Riak KV is key-value oriented. Attribute searches and set/cart logic may require application-level code or supported Riak features.

• Neo4j: nodes represent entities, relationships connect entities, and Cypher is the query language.

• Expected outputs are sample outputs, not captured results from a running server. Use the endpoint, credentials, and syntax supported by your installed version.